

CHEMS-R-US

Better living through Chemistry

Kristin Bennett, Lis Brown and John Erickson

April 2026

1.0 Chems-R-Us Task

- The European REACH regulation requires information on ready biodegradation, which is a screening test to assess the biodegradability of chemicals. At the same time REACH encourages the use of alternatives to animal testing.
- Your contract is to build a classification model to predict ready biodegradation of chemicals and to predict the classification of 400 newly developed molecules. This will help Chems-R-Us to design new materials with desired biodegradation properties more quickly and for lower cost.

Chems-R-Us Challenge Data Files

- TRAINING DATA is divided into four files:
 - `chems_train.data.csv`: Training data matrix with no response labels (1018 samples x 168 feature values)
 - `chems_feat.name.csv`: Names of the 168 attributes (168 x 1 features names)
 - `chems_train.solution.csv`: Training target values (1018 lines x 1 column)
- EXTERNAL TEST DATA is one file
 - `chems_test.data.csv`: Test data matrix (437 samples x 168 features values)

Chems-R-Us Baseline LR Methodology: Beat This!

1. Read in the data; create `train` and `validation` datasets
2. Train Logistic Regression (LR) on the training set and evaluate balanced accuracy on the validation set.
3. Review LR results to find features flagged as significant, with p-values of less than 0.2.
4. Create a new LR using only those features; evaluate balanced accuracy on the validation set.
5. Pick the best feature set by comparing the balanced accuracies and AUC on validation
6. Create a file with numbers for each test point ranking biodegradability of test molecules.
7. Create a file with 0 and 1 indicating selected features selected.
8. Prepare a `.zip` file containing **two files**, one with the testing set rankings and one indicating selected features
9. Upload this zip to the Challenge portal (details follow)

Reading the Data

```
# Prepare biodegradability data
# get feature names
featurenames <- read.csv("~/MATP-4400/data/chems_feat.name.csv",
                        header=FALSE,
                        colClasses = "character")

# get training data and rename with feature names
cdata.df <- read.csv("~/MATP-4400/data/chems_train.data.csv",
                    header=FALSE)
colnames(cdata.df) <- featurenames$V1

# get external testing data and rename with feature names
tdata.df <- read.csv("~/MATP-4400/data/chems_test.data.csv",
```

```

        header=FALSE)

colnames(tdata.df) <- featurenames$V1

class <- read.csv("~/MATP-4400/data/chems_train.solution.csv",
                 header=FALSE,
                 colClasses = "factor")

class <- class$V1

```

Preparing the Data: Create Training & Validation datasets

We split the data into **90% train** and **10% validation** datasets.

```

#ss will be the number of data points in the training set
n <- nrow(cdata.df)
ss <- ceiling(n*0.90)

# Set random seed for reproducibility
set.seed(300)
train.perm <- sample(1:n,ss)

#Split training and validation data
train <- cdata.df %>% dplyr::slice(train.perm)
validation <- cdata.df %>% dplyr::slice(-train.perm)

```

Preparing the Data: Create Training & Validation datasets (2)

We then standardize all variables to a *common scale*, since we don't have domain knowledge and LR assumes independent and identically distributed (IID) Gaussian features. This avoids LR unwittingly prioritizing certain features simply because their scale is larger.

Note that the training, validation sets, are all scaled identically using the `preProcess()`.

```

# Initialize the 'scaler' on the training data
# method = "center" subtracts the mean of the predictor's data from the predictor values
# method = "scale" divides by the standard deviation.
# NOTE: See '?preProcess' for other methods
scaler <- preProcess(train, method = c("center", "scale"))

# Use the 'scale' object to normalize our training data
train <- predict(scaler, train)
#summary(train[,1:4])

# Normalize validation data
validation <- predict(scaler, validation)

# Normalize testing data
test <- predict(scaler, tdata.df)

# Split the output classes
classtrain <- class[train.perm]
classval <- class[-train.perm]

# Convert predictions into classes

prob_to_class <- function(SVMfit,threshold=0.5) {
  # This helper function converts LR probability outputs into 1 and -1 classes
  temp <- SVMfit > threshold
  temp[temp==TRUE] <- 1
  temp[temp==FALSE] <- -1
  return(as.factor(temp))
  class(temp)
}

```

Fit XGBoost with Feature Selection

Run XGBoost, get a ranking of important features, strike the ones below a specified metric to do feature selection. Strike ones that overfit the data. Then fit on XGBoost and LDA

```

train.matrix <- as.matrix(train)

# pretty default parameters observed from googling and looking at libraries
param <- list(max.depth = 5,
              nthread = 1,
              objective = "binary:logistic",
              eval_metric = "auc")

# run a longer, in depth version where features are better explored
XGfit.unfiltered <- xgboost(data = train.matrix, label = classtrain, params = param,
                           eta = 1, nround = 500, verbose = 1)
XGfit.unfiltered.train <- predict(XGfit.unfiltered, train, type = "response")
XGfit.unfiltered.val <- predict(XGfit.unfiltered, validation, type = "response")
feature.importance <- xgb.importance(XGfit.unfiltered)

# training and label data is train and classtrain respectively
# we are doing a binary logistic regression, but this time with XGBoost
# values given are pretty default values based on documentation
# shorter run, grab less values and trends, meant for
#spotting overfitted features
XGfit <- xgboost(data = train.matrix, label = classtrain, params = param,
                eta = 1, nround = 50, verbose = 1)
XGfit.train <- predict(XGfit, train.matrix, type = "response")
XGfit.val <- predict(XGfit, validation, type = "response")

feature.importance.removal <- xgb.importance(XGfit)

# there may be a feature that is extremely overfitted. This code identifies
#that feature and removes it
overfitFeature <- feature.importance$Feature[1]
if (feature.importance$Gain[1] > 0.5){
  train.matrix <- train.matrix[, colnames(train.matrix) !=
                                feature.importance$Feature[1]]
}

```

```

# run a longer, in depth version where features are better explored
XGfit <- xgboost(data = train.matrix, label = classtrain, params = param,
               eta = 1, nround = 500, verbose = 1)
XGfit.train <- predict(XGfit, train, type = "response")
XGfit.val <- predict(XGfit, validation, type = "response")
feature.importance <- xgb.importance(XGfit)

# logistic feature selection, initially all false
significant.variables <- rep(FALSE, 168)

for(i in 1:size(feature.importance$Gain)[2]){
  # cut off about half of the features, a balance?
  if(feature.importance$Gain[i] > 0.0005){
    # find the column associated with this feature
    significant.variables[which(colnames(train.matrix)==
                              feature.importance$Feature[i])] = TRUE
  }
}

# significant.variables <- as.matrix(significant.variables)
significant.variables <- t(significant.variables)
colnames(significant.variables) <- colnames(train)
significant.variables <- t(significant.variables)

# Keep only statistically significant features
train.fs <- train %>% select_if(significant.variables)
train.fs.df <- cbind(train.fs, classtrain)
train.fs.matrix <- as.matrix(train.fs)

# same with validation

# significant.variables <- as.matrix(significant.variables)
significant.variables <- t(significant.variables)
colnames(significant.variables) <- colnames(validation)
significant.variables <- t(significant.variables)

# Keep only statistically significant features
validation.fs <- validation %>% select_if(significant.variables)
validation.fs.df <- cbind(validation.fs, classval)
validation.fs.matrix <- as.matrix(validation.fs)

# run a longer in depth XGBoost
XGfit.fs <- xgboost(data = train.fs.matrix, label = classtrain,
                  params = param, eta = 1, nround = 500, verbose = 1)
XGfit.fs.train <- predict(XGfit.fs, train.fs.matrix, type = "response")
XGfit.XGfs.val <- predict(XGfit.fs, validation, type = "response")

# calculated balanced accuracy
XGfit.fs.val.class <- prob_to_class(XGfit.XGfs.val)
XGfit.val.class <- prob_to_class(XGfit.XGfs.val)
result<-confusionMatrix(classval,XGfit.val.class)
confusion.matrix.result <- confusionMatrix(classval, XGfit.fs.val.class)
BalancedAccuracyLRFS <- confusion.matrix.result$byClass["Balanced Accuracy"]

# format and export results

# Predict the test data (OUTPUTS LOG-ODDS) to get rankings
ranking_lrtest <- predict(XGfit.fs, test)
ranking_lrtest <- as.numeric(ranking_lrtest)

# no need to convert to 0 and 1 since ranking needed for AUC.
write.table(ranking_lrtest,file =
  "classification.csv", row.names=F, col.names=F)

# Here is the mean prediction file for submission to the website
# features should be a column vector of 0's and 1's.
# 1 = keep feature, 0 = don't
features<-matrix(0,nrow=(ncol(train)),ncol=1)
rownames(features) <- colnames(train)

# Set the ones we want to keep to 1
features[significant.variables,1] <- 1
write.table(features,file = "selection.csv", row.names=F, col.names=F)

# fit XGRanking on lda
# poor accuracy likely due to mediocre feature selection from the get-go
LDA.fs <- lda(train.fs.matrix, classtrain)
LDAfit.fs.train <- predict(LDA.fs, train.fs.matrix, type = "response")
LDAfit.LDAfs.val <- predict(LDA.fs, validation.fs.matrix, type = "response")

test.matrix <- as.matrix(test)
significant.variables <- t(significant.variables)

test.matrix <- rbind(test.matrix,significant.variables)
test.matrix.t <- t(test.matrix)
test.matrix.t.df <- as.data.frame(test.matrix.t)
test.matrix.t.df <- subset(test.matrix.t.df, significant.variables == 1)
test.matrix.t.df <- subset(test.matrix.t.df, select = ~significant.variables)
test.matrix.t <- as.matrix(test.matrix.t.df)
test.matrix <- t(test.matrix.t)

# Predict the test data (OUTPUTS LOG-ODDS) to get rankings
ranking_lrtest <- predict(LDA.fs, test.matrix)
#ranking_lrtest <- as.numeric(ranking_lrtest$class)

# no need to convert to 0 and 1 since ranking needed for AUC.
# rankinglr_test <-prob_to_class(ranking_lrtest)

```

```

ranking.lda <- rep(1, 400)
for(i in 1:size(ranking.lda)[2]){
  if(ranking_lrtest$class[i] == -1){
    ranking.lda[i] = -1
  }
}

write.table(ranking.lda,file =
  "classification.csv", row.names=F, col.names=F)

# Here is the mean prediction file for submission to the website
# features should be a column vector of 0's and 1's.
# 1 = keep feature, 0 = don't
features<-matrix(0,nrow=(ncol(train)),ncol=1)
rownames(features) <- colnames(train)

# Set the ones we want to keep to 1
features[significant.variables,1] <- 1
write.table(features,file = "selection.csv", row.names=F, col.names=F)

# get time
time <- format(Sys.time(), "%H%M%S")

#This automatically generates a compressed (zip) file
system(paste0("zip -u willic21Entry-", time, ".csv.zip classification.csv"))
system(paste0("zip -u willic21Entry-", time, ".csv.zip selection.csv"))

paste0("The name of your entry file: willic21Entry-", time, ".csv.zip")

```

```
## [1] "The name of your entry file: willic21Entry-202211.csv.zip"
```

LASSO Regression

Solid classification rate, totally fell apart on feature selection and I'm unsure why. From my understanding, LASSO sends unimportant values to 0 Got a value of lambda (s) that has half nonzero coefficients I must have messed something up or misunderstand how to effectively use LASSO, my feature selection score was in the 30s

```

# train.df <- cbind(train,classtrain)
train.matrix <- as.matrix(train)
validation.matrix <- as.matrix(validation)
# label is classtrain
lasso.fit <- glmnet(train.matrix, classtrain, family = "binomial")
lasso.train <- predict(lasso.fit, as.matrix(train), type = "response")
lasso.unfiltered.val <- predict(lasso.fit, as.matrix(validation), type = "response")
# plot(lasso.fit, label = TRUE)

# 0.004 gave about half nonzero coefficients
# i filtered out features that had a coefficient larger than 0.1
# since I suspected overfitting from them
# which implied 'fake' features
lasso.coef <- coef(lasso.fit, s = 0.004)
lasso.coef.matrix <- as.matrix(lasso.coef)
for(i in 2:size(lasso.coef.matrix)[1]){
  if(abs(lasso.coef.matrix[i,1]) > 0.1){
    lasso.coef.matrix[i,1] = 0
  }
}

# feature selection - anything that is a 0 = discard.
#anything that isn't 0 = keep
# class selection - positive values are in +1, negative is in -1

# feature selection, initially all false
significant.variables <- rep(FALSE, 168)

for(i in 1:size(lasso.coef.matrix)[1]-1){
  # print(lasso.coef.matrix[i,1])
  if(lasso.coef.matrix[i+1,1] != 0){
    # find the column associated with this feature
    significant.variables[i] = TRUE
  }
}

# filter out variables that are irrelevant
# basically just bind the significant variables and remove all the ones from
#the column that are irrelevant
train.matrix <- rbind(train.matrix,significant.variables)
train.matrix.t <- t(train.matrix)
train.matrix.t.df <- as.data.frame(train.matrix.t)
train.matrix.t.df <- subset(train.matrix.t.df, significant.variables == 1)
train.matrix.t.df <- subset(train.matrix.t.df, select = ~significant.variables)
train.matrix.t <- as.matrix(train.matrix.t.df)
train.fs.matrix <- t(train.matrix.t)

# need to do the same with validation
validation.matrix <- rbind(validation.matrix,significant.variables)
validation.matrix.t <- t(validation.matrix)
validation.matrix.t.df <- as.data.frame(validation.matrix.t)
validation.matrix.t.df <- subset(validation.matrix.t.df, significant.variables == 1)
validation.matrix.t.df <- subset(validation.matrix.t.df, select = ~significant.variables)
validation.matrix.t <- as.matrix(validation.matrix.t.df)
validation.fs.matrix <- t(validation.matrix.t)

# Now use XGBoost. same parameters as above.

XGfit.fs <- xgboost(data = train.fs.matrix, label = classtrain, params = param,
  eta = 1, nround = 500, verbose = 1)
XGfit.fs.train <- predict(XGfit.fs, train.fs.matrix, type = "response")
XGfit.lasso.fs.val <- predict(XGfit.fs, validation.fs.matrix, type = "response")

# write to file and zip

# Predict the test data (OUTPUTS LOG-ODDS) to get rankings

```

```

ranking_lrtest <- predict(XGfit.fs, test)
ranking_lrtest <- as.numeric(ranking_lrtest)

# no need to convert to 0 and 1 since ranking needed for AUC.
rankinglr_test <-prob_to_class(ranking_lrtest)

write.table(ranking_lrtest,file =
            "classification.csv", row.names=F, col.names=F)

# Here is the mean prediction file for submission to the website
# features should be a column vector of 0's and 1's.
# 1 = keep feature, 0 = don't
features<-matrix(0,nrow=(ncol(train)),ncol=1)
rownames(features) <- colnames(train)

# Set the ones we want to keep to 1
features[significant.variables,1] <- 1
write.table(features,file = "selection.csv", row.names=F, col.names=F)

# get time
time <- format(Sys.time(), "%H%M%S")

#This automatically generates a compressed (zip) file
system(paste0("zip -u willic21Entry-", time, ".csv.zip classification.csv"))
system(paste0("zip -u willic21Entry-", time, ".csv.zip selection.csv"))

paste0("The name of your entry file: willic21Entry-", time, ".csv.zip")

## [1] "The name of your entry file: willic21Entry-203003.csv.zip"

# fit lasso on lda
# poor accuracy likely due to bad feature selection from the get-go
LDA.fs <- lda(train.fs.matrix, classtrain)
lda.fs.lasso.train <- predict(LDA.fs, train.fs.matrix, type = "response")
lda.fs.lasso.val <- predict(LDA.fs, validation.fs.matrix, type = "response")

test.matrix <- as.matrix(test)
#validation.matrix <- as.matrix(validation)

test.matrix <- rbind(test.matrix,significant.variables)
test.matrix.t <- t(test.matrix)
test.matrix.t.df <- as.data.frame(test.matrix.t)
test.matrix.t.df <- subset(test.matrix.t.df, significant.variables == 1)
test.matrix.t.df <- subset(test.matrix.t.df, select = ~significant.variables)
test.matrix.t <- as.matrix(test.matrix.t.df)
test.matrix <- t(test.matrix.t)

# Predict the test data (OUTPUTS LOG-ODDS) to get rankings
ranking_lrtest <- predict(LDA.fs, test.matrix)
#ranking_lrtest <- as.numeric(ranking_lrtest$class)

# no need to convert to 0 and 1 since ranking needed for AUC.
# rankinglr_test <-prob_to_class(ranking_lrtest)

ranking.lda <- rep(1, 400)
for(i in 1:size(ranking.lda)[2]){
  if(ranking_lrtest$class[i] == -1){
    ranking.lda[i] = -1
  }
}

write.table(ranking.lda,file =
            "classification.csv", row.names=F, col.names=F)

# Here is the mean prediction file for submission to the website
# features should be a column vector of 0's and 1's.
# 1 = keep feature, 0 = don't
features<-matrix(0,nrow=(ncol(train)),ncol=1)
rownames(features) <- colnames(train)

# Set the ones we want to keep to 1
features[significant.variables,1] <- 1
write.table(features,file = "selection.csv", row.names=F, col.names=F)

# get time
time <- format(Sys.time(), "%H%M%S")

#This automatically generates a compressed (zip) file
system(paste0("zip -u willic21Entry-", time, ".csv.zip classification.csv"))
system(paste0("zip -u willic21Entry-", time, ".csv.zip selection.csv"))

paste0("The name of your entry file: willic21Entry-", time, ".csv.zip")

## [1] "The name of your entry file: willic21Entry-202213.csv.zip"

```

Using Bourta with XGBoost/LDA

Slow to fit but highest feature accuracy

```

train.matrix <- as.matrix(train)
validation.matrix <- as.matrix(validation)
boruta.fit <- Boruta(train.matrix, classtrain)

# convert final decision to numeric
boruta.fit.numeric <- as.numeric(boruta.fit$finalDecision)

# convert into significant variables format

# feature selection, initially all false
significant.variables <- rep(FALSE, 168)

for(i in 1:size(significant.variables)[2]){

```

```

    if(boruta.fit.numeric[i] != 3){ # 3 means 'reject'
      significant.variables[i] = TRUE
    }
  }

  # reusing this from before - we identified an overfitted feature earlier
  # we remove this feature
  # lower feature selection score (since X160 is somehow one of the good ones)
  # but much better classification score
  # leading to better balanced accuracy
  # basically two feature selections done in one go - XC potential for creativity?

  # identify any overfitted features, if any. otherwise skip this part

overfitRemoval <- colnames(train.matrix) != feature.importance.removal$Feature[1]
overfitRemoval <- as.numeric(overfitRemoval)

#overfitFeature <- feature.importance.removal$Feature[1]
#if (feature.importance.removal$Gain[1] > 0.5){
#  for(i in 1:size(significant.variables)[2]){
#    if (overfitRemoval[i] == 0){
#      significant.variables[i] = FALSE
#    }
#  }
#}
#}

# filter out variables that are irrelevant
train.matrix <- rbind(train.matrix,significant.variables)
train.matrix.t <- t(train.matrix)
train.matrix.t.df <- as.data.frame(train.matrix.t)
train.matrix.t.df <- subset(train.matrix.t.df, significant.variables == 1)
train.matrix.t.df <- subset(train.matrix.t.df, select = ~significant.variables)
train.matrix.t <- as.matrix(train.matrix.t.df)
train.fs.matrix <- t(train.matrix.t)

validation.matrix <- rbind(validation.matrix,significant.variables)
validation.matrix.t <- t(validation.matrix)
validation.matrix.t.df <- as.data.frame(validation.matrix.t)
validation.matrix.t.df <- subset(validation.matrix.t.df, significant.variables == 1)
validation.matrix.t.df <- subset(validation.matrix.t.df, select = ~significant.variables)
validation.matrix.t <- as.matrix(validation.matrix.t.df)
validation.fs.matrix <- t(validation.matrix.t)

# fit around xgboost

# default

XGfit.fs <- xgboost(data = train.fs.matrix, label = classtrain, params = param,
  eta = 1, nround = 500, verbose = 1)
XGfit.fs.train <- predict(XGfit.fs, train.fs.matrix, type = "response")
XGfit.boruta.fs.val <- predict(XGfit.fs, validation.fs.matrix, type = "response")

# prepare for submission
# write to file and zip

# Predict the test data (OUTPUTS LOG-ODDS) to get rankings
ranking_lrtest <- predict(XGfit.fs, test)
ranking_lrtest <- as.numeric(ranking_lrtest)

# no need to convert to 0 and 1 since ranking needed for AUC.
rankinglr_test <- prob_to_class(ranking_lrtest)

write.table(ranking_lrtest,file =
  "classification.csv", row.names=F, col.names=F)

# Here is the mean prediction file for submission to the website
# features should be a column vector of 0's and 1's.
# 1 = keep feature, 0 = don't
features<-matrix(0,nrow=(ncol(train)),ncol=1)
rownames(features) <- colnames(train)

# Set the ones we want to keep to 1
features[significant.variables,1] <- 1
write.table(features,file = "selection.csv", row.names=F, col.names=F)

# get time
time <- format(Sys.time(), "%H%M%S")

#This automatically generates a compressed (zip) file
system(paste0("zip -u willic21Entry-", time, ".csv.zip classification.csv"))
system(paste0("zip -u willic21Entry-", time, ".csv.zip selection.csv"))

paste0("The name of your entry file: willic21Entry-", time, ".csv.zip")

## [1] "The name of your entry file: willic21Entry-203004.csv.zip"

# fit boruta on lda
LDA.fs <- lda(train.fs.matrix, classtrain)
LDA.fs.train <- predict(LDA.fs, train.fs.matrix, type = "response")
LDA.boruta.fs.val <- predict(LDA.fs, validation.fs.matrix, type = "response")

test.matrix <- as.matrix(test)

test.matrix <- rbind(test.matrix,significant.variables)
test.matrix.t <- t(test.matrix)
test.matrix.t.df <- as.data.frame(test.matrix.t)
test.matrix.t.df <- subset(test.matrix.t.df, significant.variables == 1)
test.matrix.t.df <- subset(test.matrix.t.df, select = ~significant.variables)
test.matrix.t <- as.matrix(test.matrix.t.df)
test.matrix <- t(test.matrix.t)

# Predict the test data (OUTPUTS LOG-ODDS) to get rankings
ranking_lrtest <- predict(LDA.fs, test.matrix)

```

```

#ranking_lrtest <- as.numeric(ranking_lrtest$class)

# no need to convert to 0 and 1 since ranking needed for AUC.
# ranking_lr_test <- prob_to_class(ranking_lrtest)

ranking_lda <- rep(1, 400)
for(i in 1:size(ranking_lda)[2]){
  if(ranking_lrtest$class[i] == -1){
    ranking_lda[i] = -1
  }
}

write.table(ranking_lda, file =
  "classification.csv", row.names=F, col.names=F)

# Here is the mean prediction file for submission to the website
# features should be a column vector of 0's and 1's.
# 1 = keep feature, 0 = don't
features <- matrix(0, nrow=(ncol(train)), ncol=1)
rownames(features) <- colnames(train)

# Set the ones we want to keep to 1
features[significant.variables, 1] <- 1
write.table(features, file = "selection.csv", row.names=F, col.names=F)

# get time
time <- format(Sys.time(), "%H%M%S")

# This automatically generates a compressed (zip) file
system(paste0("zip -u willic21Entry-", time, ".csv.zip classification.csv"))
system(paste0("zip -u willic21Entry-", time, ".csv.zip selection.csv"))

paste0("The name of your entry file: willic21Entry-", time, ".csv.zip")

## [1] "The name of your entry file: willic21Entry-202307.csv.zip"

```

Classification Challenge

- Chems-R-Us would like a “ranking” of the testing molecules in terms of bioavailability.
- The ranking can be any number *leading to a classification*. Here we like *log odds*.
- This means **larger values are more likely to be in class 1** and **values less than 0 mean class -1**.
- The results will be ranked by AUC.

Computing ROC Curves for LR & LR with Feature Selection

Here we compute the ROC curves for LR and LR with feature selection. We can see that their performance is very comparable.

- **Sensitivity** (also called the *true positive rate* or the *recall*) measures the proportion of actual positives that are correctly identified as such (e.g., the percentage of passengers that survived people who are correctly identified as having survived). *It is the same as the Class 1 accuracy.*
- **Specificity** (also called the *true negative rate*) measures the proportion of actual negatives that are correctly identified as such (e.g., the percentage of patients who died who are correctly identified as not having died). *It is the same as the Class -1 accuracy.*

ROC Curves (code)

```

roc.data <- data.frame("Class" = classval,
  "Selection - XGFit" = XGfit.XGfs.val,
  "Selection - XGfit Ranking + LDA" = LDAfit.LDAfs.val$posterior[,2],
  "Selection - LASSO + LDA" = lda.fs.lasso.val$posterior[,2],
  "Selection - LASSO + fit" = XGfit.lasso.fs.val,
  "Selection - Boruda + LDA" = LDA.boruta.fs.val$posterior[,2],
  "Selection - Boruda + XG" = XGfit.boruta.fs.val)

##"No Selection - XGFit")

# Use pROC::roc to construct the ROC object (but not to plot!)
roc.list <- roc(Class ~.,
  data = roc.data)

#no.selection.auc <- round(auc(Class ~ With.Selection, data = roc.data), digits = 3)
with.selection.auc <- round(auc(Class ~ Selection..XGFit, data = roc.data), digits = 3)
with.selection.xglda.auc <- round(auc(Class ~ Selection..XGfit.Ranking...LDA, data = roc.data), digits = 3)
with.selection.lassolda.auc <- round(auc(Class ~ Selection...LASSO...LDA, data = roc.data), digits = 3)
with.selection.lassoxg.auc <- round(auc(Class ~ Selection...LASSO...fit, data = roc.data), digits = 3)
with.selection.borudalda.auc <- round(auc(Class ~ Selection...Boruda...LDA, data = roc.data), digits = 3)
with.selection.borudaxg.auc <- round(auc(Class ~ Selection...Boruda...XG, data = roc.data), digits = 3)

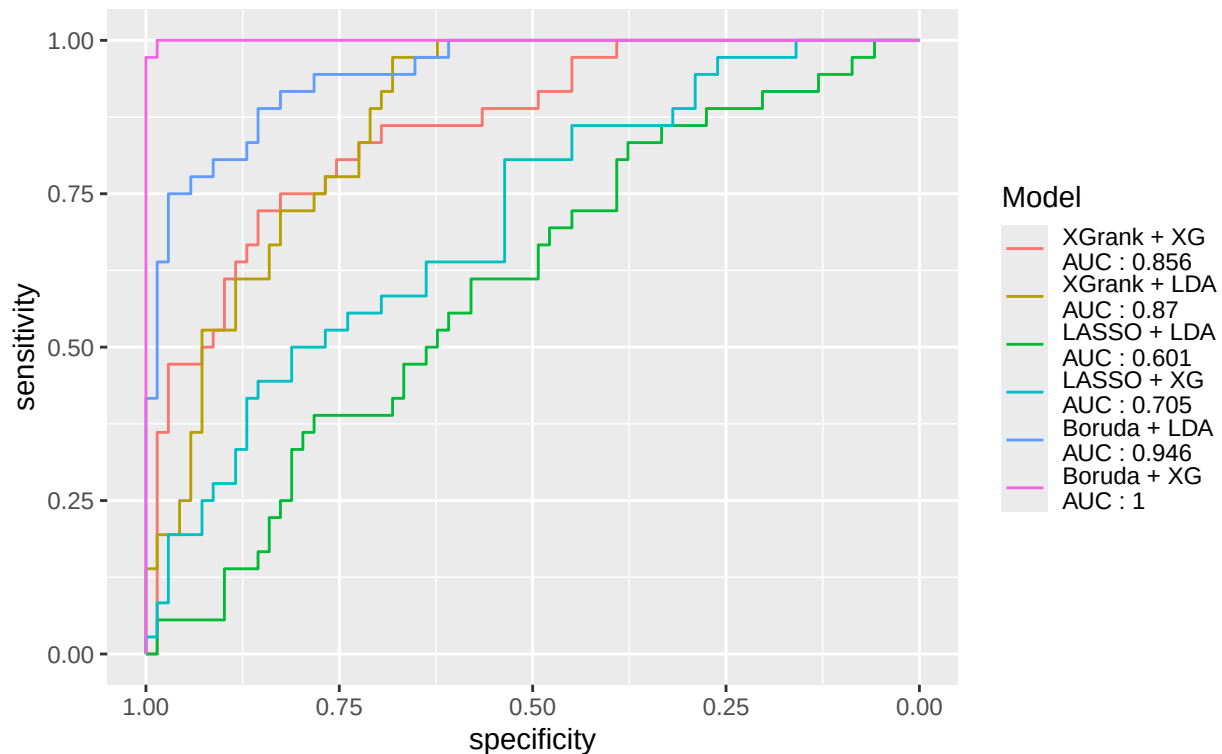
# Use pROC::ggroc (based on ggplot2) for the actual plot
roc_plot <- ggroc(roc.list) +
  ggtitle("ROC Curves (Validation Set)", subtitle = "All feature selections compared") +
  scale_color_discrete(name = "Model",
    labels = c(paste("XGrank + XG\nAUC :", with.selection.auc),
      paste("XGrank + LDA\nAUC :", with.selection.xglda.auc),
      paste("LASSO + LDA\nAUC :", with.selection.lassolda.auc),
      paste("LASSO + XG\nAUC :", with.selection.lassoxg.auc),
      paste("Boruda + LDA\nAUC :", with.selection.borudalda.auc),
      paste("Boruda + XG\nAUC :", with.selection.borudaxg.auc)))

roc_plot

```

ROC Curves (generated plot)

ROC Curves (Validation Set)
All feature selections compared



Contest results

I hope this suffices as a table?

XGBoost + Boruda = 0.9215 balanced, 0.9025 classification, 0.9405 feature LDA + Boruda = 0.8706 balanced, 0.8242 classification, 0.9286 feature XGBoost + XGBoost ranking = 0.7352 balanced, 0.8832 classification, 0.5873 feature LDA + XGBoost ranking = 0.5910 balanced, 0.8686 classification, 0.3135 feature XGBoost + LASSO = 0.4503 balanced, 0.5593 classification, 0.3413 feature LDA + LASSO = 0.4272 balanced, 0.5331 classification, 0.3214 feature

Final thoughts + Conclusion

(note that I wrote this conclusion to count for several rubric criteria, including analysis of features)

The best results came when Boruda feature selection was used. This is a highly accurate feature selection and I believe it is responsible for all the top feature selection scores in the class. When the feature selection was accurate, the classification scores on LDA and XGBoost went way up.

XGBoost always fared better than LDA. I think there's just a lot more control of how it works and is naturally just a better fit than LDA. There's a bit of inconsistency with that on the ROC curve, idk if I just did it wrong or if I misremembered which numbers were associated with test results. Point is that XGBoost tends to yield better results.

I'm unsure as to why there are occasionally huge differences between feature selections. I might have changed something that yielded inaccurate results (for example, the LDA + XGBoost ranking should have the same feature accuracy, yet they are different.)

Lasso was just downright ineffective. I have heard from others that they didn't have much luck with it either. I might have set something up incorrectly. But anyway, the poor feature selection correlated to poor classification.

I talked to my friend who happened to get the top result in the entire class. He said that XGBoost can overfit on small datasets, so he used random forest instead in combination with the boruda that I did on my best submission. Perhaps this could work better and not require me to remove features to make the balanced accuracy go up. Perhaps a record breaking classification score could be achieved if the very overfitted feature, X160, was removed (though this appears to ironically be one of the 'good' variables; removing it in testing resulting in lower contest AUC scores.)

Tips for Success

- Warning many feature selection methods require the data to be scaled to work well. Make sure you use the train mean and standard deviations for scaling all other datasets.
- If you get balanced accuracy of 0.5 or less than 0, then there is probably a mistake in the format of the entry that you submitted. Check it carefully.
- Don't wait until the last minute to start entering the challenge. Get some solution working and then refine. Get help early if you are getting funny answers on the challenge. It is usually easy to fix if you get help.
- If your ROC curve is a straight line with one or two kinks in it, then you are doing the ranking wrong. You have to have your class prediction return a probability estimate of the positive class or a linear function value. If you do it correctly, then the ROC curve will be relatively smooth.
- How you make the ranking can be different for a specific method. Understand the outputs of predict for that method and construct the appropriate ranking values (usually probabilities of positive class).

I, Connor Williams, confirm that I did not use GenAI to complete any aspect of this project.